INSTITUT TEKNOLOGI DEL

MATERI TUGAS

Pemrograman Berorientasi Objek

SEMESTER GANJIL T.A 2026/2027

Tanggal / Sesi	:	Rabu, 21 Oktober 2026
Semester	:	Ganjil
Mata Kuliah	:	Pemrograman Berorientasi Objek
Minggu/Sesi	:	07/ 02
Aktivitas	:	Exception Handling
Durasi	:	110 Menit
Setoran	:	Softfile
Batas Pengumpulan	:	Akhir Sesi Praktikum
Tujuan	:	Mahasiswa mampu memahami dan menguasai tentang Exception Handling

---

Exeption Handling

Coba jawab semua pertanyaan yang muncul bagian-bagian langkah praktikum ini.

1. Uncaught Exception

```
class UncaughtException {
    public static void main(String args[]) {
        System.out.println(3/0);
    }
}
```

   a. Analisis apa yang akan terjadi ketika program di atas di eksekusi.
   b. Apa outputnya?

2. Handling the Excception

```
public class HandlingException {
    public static void main(String[] args) {
        try {
            System.out.println(3/0);
        } catch (ArithmeticException ex) {
            System.out.println("Error " + ex);
        }
    }
}
```

   a. Ganti ex dengan ex.getMessage() dan ex.getStackTrace(). Coba juga untuk mengeksekusi statement ex.printStackTrace() dan lihat apa hasilnya.
   b. Ganti AtirhmeticException dengan Exception. Apa hasilnya? Jelaskan.

3. Using Finally

```
class CaughtExceptionFinally {
    public static void main(String args[]) {
        int n = args.length;
        try {
            if (n == 0) {
                System.out.println(3/n);
                System.out.println("This won't be executed");
            } else {
                System.out.println("This will be executed if "
                        + "arguments is not zero");
            }
        } catch (Exception e) {
            System.out.println("Catching exception: " + e);
        } finally {
            System.out.println("After trying or catching...");
        }
    }
}
```

   a. Coba eksekusi program di atas tanpa arguments. Apa hasilnya?
   b. Coba eksekusi program di atas dengan arguments. Apa hasilnya?
   c. Jelaskan kesimpulan Anda dari pertanyaan a dan b setelah menjalankan program.

4. Modifikasi kode program no.3. Ganti Exception di dalam catch block dengan NullPointerException. Eksekusi dan jalankan kode. Apa hasilnya dan jelaskan apa yang terjadi jika sebuah exception tidak memiliki sebuah catch yang cocok?

5. Multiple Catch

```
class MultipleCatch {
    public static void main(String args[]) {
        int n = args.length;
        int a = 1;
        int c[] = new int[1];
        try {
            if (n == 0)
                a = a / (a - 1);
            else
                c[21] = 5;
        } catch (ArithmeticException e) {
            System.out.println("Ups, seem out we have an Arithmetic "
                    + "Exception here...");
        } catch (Exception e) {
            System.out.println("Let the other exception handler here...");
        }
    }
}
```

a. Coba eksekusi program tanpa arguments. Catch statement mana yang dieksekusi?

b. Coba eksekusi program dengan sebuah argument. Catch statement mana yang dieksekusi?

c. Sekarang, ganti ArithmeticException dengan Excception sepeti yang ditunjukkan di bawah ini:

```
..
} catch (Exception e){
    System.out.println("Ups, seem out we have an Arithmetic Exception
here...");
} catch (ArithmeticException e){
    System.out.println("Let the other exception handler here...");
} ..
```

Apa yang terjadi? Mengapa? Jelaskan

6. Throwing an Exception

```
class ThrowExc {
    public static void main(String args[]) {
        try {
            System.out.println("I'm throwing an exception from here...");
            throw new NullPointerException("throw demo");
        } catch (NullPointerException e) {
            System.out.println("I'm catching an exception: [" + e + "] "
                    + "from here...");
        }
    }
}
```

7. Re-throwing an Excception

```
class ReThrowExc {
    static void demoThrow() {
        try {
            System.out.println("I'm throwing an exception from demoThrow "
                    + "method");
            throw new NullPointerException("demo");
        } catch (NullPointerException e) {
            System.out.println("I'm catching the exception [" + e + "]");
            System.out.println("But I'll throw it again...\n");
            throw e;
        }
    }

    public static void main(String args[]) {
        try {
            demoThrow();
        } catch (NullPointerException e) {
            System.out.println("I'm catching the exception [" + e + "]\n "
                    + "thrown from demoThrow method");
        }
    }
}
```

a. Jika exception e tidak dilempar (rethrown) dari method demoThrow, apakah masih akan di tangkap dari dalam main function?
b. Apa yang bisa Anda simpulkan dari kode program di atas?

8. Throws Demo

```
class ThrowsDemo {
    static void throwOne(){
        System.out.println("I'm throwing a non Runtime exception that I "
                + "don't handle");
        System.out.println("Without specifying in method declaration");
        throw new IllegalAccesException("demo");
    }

    public static void main (String args[]){
        throwOne();
    }

}
```

a. Apa yang akan terjadi jika kode program di atas dieksekusi? Mengapa?
b. Apa yang akan terjadi jika IllegalAccessException diganti dengan ArithmeticException? Mengapa? Perbaiki kode program di atas seperti yang ditunjukkan di bawah ini:

```
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        System.out.println("I'm throwing a non Runtime exception that I "
                + "don't handle");
        System.out.println("Without specifying in method declaration");
        throw new IllegalAccessException("demo");
    }

    public static void main(String args[]) {
        try {
            throwOne();
        } catch (IllegalAccessException e) {
            System.out.println("Caught " + e);
        }
    }
}
```

9. Membuat exception sendiri.

```
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        System.out.println("I'm throwing a non Runtime exception that I "
                + "don't handle");
        System.out.println("Without specifying in method declaration");
        throw new IllegalAccessException("demo");
    }

    public static void main(String args[]) {
        try {
            throwOne();
        } catch (IllegalAccessException e) {
            System.out.println("Caught " + e);
        }
    }
}
```

Tuliskan hasil pengamatan Anda pada kode program yang Anda kerjakan dalam bentuk line comment. Tambahkan pula pemahaman Anda tentang exception handling setelah menjalankan kode program.

Seluruh file hasil pengerjaan praktikum di atas harus **disetor ke ecourse**. Kompres file projek dengan ekstensi .zip. Nama file dengan format: **PBO_w7s02_NIM.zip**. NIM diganti dengan NIM Anda.